

SWE573 - Software Development Practice

Project Development Log - Local Craft Discovery Guide

Iteration 1: Repository, Git workflow, GitHub project management, and requirements elicitation

Student: Anil Basaran Program: Software Engineering Master's Program, Boğaziçi University
Date: 29 September 2026

Purpose

This document records the hands-on work completed during the first iteration of the SWE573 project. Commands were executed locally, their outputs were inspected, GitHub project-management features were configured, and project decisions were recorded through issues and comments. The document is a dated learning and development log, not a replacement for the repository Wiki.

1. Project concept

The course domain is indigenous and handmade crafts. Three directions were considered: an information/blog website, a general map-based craft application, and a focused discovery guide. The selected concept is Local Craft Discovery Guide: a location-based application for travelers interested in culturally significant local crafts, their background, where they can be found, and community ratings/reviews.

The concept is inspired by the focused discovery approach of the Michelin Guide, applied to local crafts rather than restaurants and cuisine.

2. Repository setup and README

The repository was cloned locally. An early mistake became a useful Git lesson: `git status` was initially run from the parent directory and returned 'not a git repository'. Changing into the cloned repository directory resolved the problem.

```
git clone https://github.com/AnilBASARAN/SWE573-SoftwareDevelopmentAssignment.git
cd SWE573-SoftwareDevelopmentAssignment
git status
```

README.md was expanded from a minimal placeholder into a project description containing the concept, problem, target users, and project goals.

3. Git workflow learned through README commits

`git diff` was used before staging to inspect changes. The first README commit added the initial project description; the second added project goals.

```
git diff
git add README.md
git commit -m "Add initial project description to README"
git commit -m "Add project goals to README"
git log --oneline

9fdeb7d (HEAD -> main) Add project goals to README
0587d5d (origin/main, origin/HEAD) Add initial project description to README
a32afbe Initial commit
```

This output made the local/remote distinction visible. HEAD -> main pointed to the local branch, while origin/main represented the locally known remote branch position. Before pushing, the local branch was one commit ahead.

```
Your branch is ahead of 'origin/main' by 1 commit.
```

After git push origin main, HEAD, main, and origin/main pointed to the same commit. The practical workflow learned was edit -> diff -> add -> commit -> push.

4. GitHub repository structure and Wiki

Version-controlled files such as .github/ISSUE_TEMPLATE and .github/workflows appear locally after cloning. GitHub Issues, labels, milestones, and project boards are platform metadata rather than ordinary repository files. GitHub Wiki is associated with the repository in the UI but is maintained as a separate Git repository.

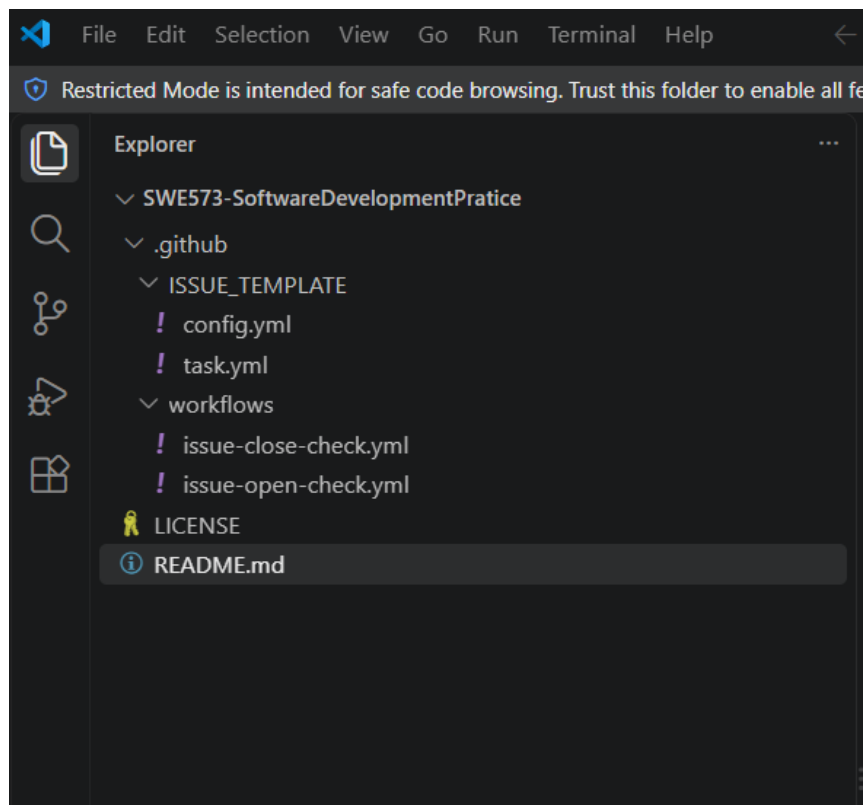


Figure 1. Example repository structure containing version-controlled .github files.

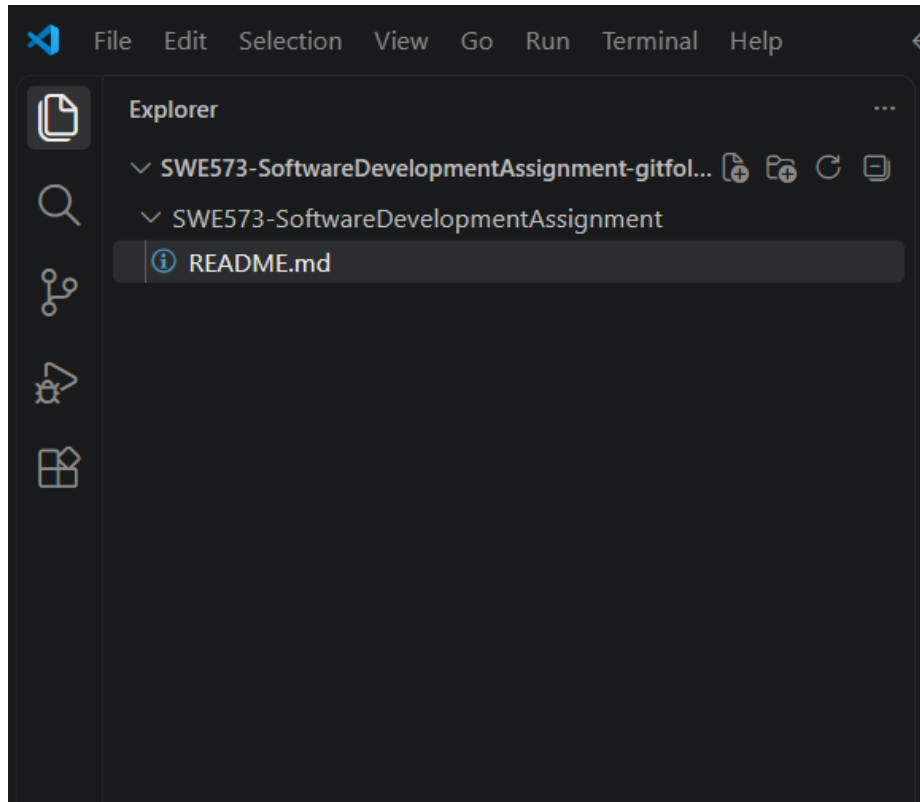


Figure 2. Local SWE573 repository early in the exercise.

5. Custom issue labels

Existing labels were reviewed and a small project-specific scheme was designed. research identifies investigation/comparison work. Priority is represented separately with priority: high, priority: medium, and priority: low. This separates the type of work from its urgency/importance.

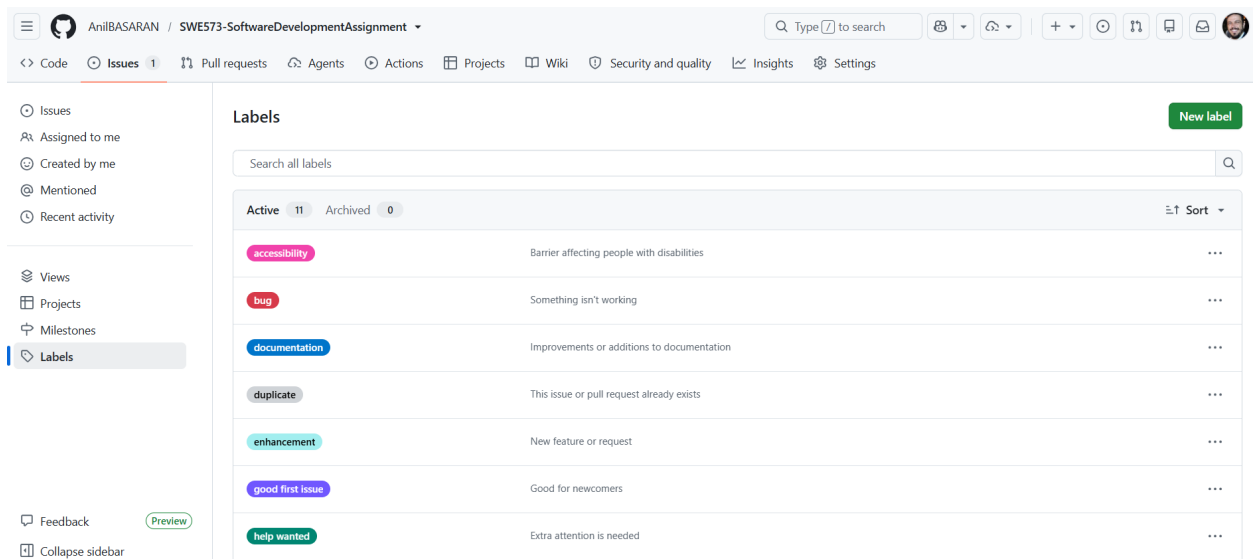


Figure 3. Reviewing the repository's existing labels.

Figure 4. Creating a custom GitHub label.

6. Issue #1 - Define the application concept

Issue #1 was refined into [TASK]: Define the application concept. Because the course domain was already indigenous crafts, the task was not to choose a domain but to decide what application should be built within it.

Acceptance criteria covered exploring alternatives, identifying the problem and target users, selecting one concept, and documenting it in README. The issue was assigned, added to the Project Planning milestone, and labeled research and priority: high.

The final decision comment documented the alternatives and the rationale for selecting Local Craft Discovery Guide.

Figure 5. Issue #1 after refining its scope and acceptance criteria.

Figure 6. Completed Issue #1 with decision history and satisfied acceptance criteria.

Issue #2 was created as [TASK]: Prepare a potential user survey for requirements elicitation. It was treated as high priority because the application's initial requirements and feature priorities should be informed by potential-user feedback rather than only assumptions.

The target group was defined as people who travel regularly and enjoy exploring local culture, including people interested in discovering handmade products or purchasing souvenirs that represent the places they visit.

SWE573 - Local Craft Discovery Guide | Project Development Log

Figure 7. Issue #2 with labels, milestone, assignee, and measurable acceptance criteria.

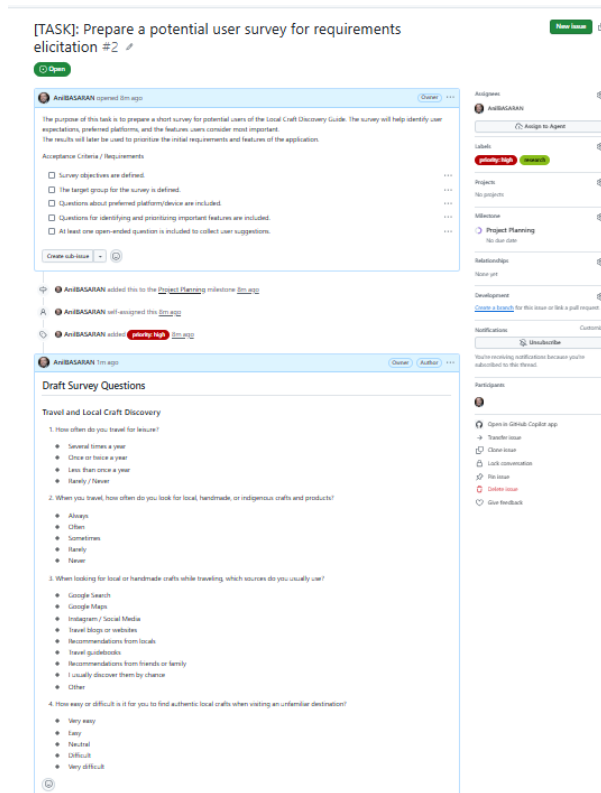


Figure 8. Draft survey questions recorded as issue comments.

8. Adding project documentation as a tracked file

A docs/ folder was created locally and the first PDF development log was placed inside it. Running git status showed the folder as untracked. After staging the PDF, Git reported it as a new file ready to be committed.

```
git status
Untracked files:
docs/

git add docs/SWE573_Project_Development_Log_Iteration_1.pdf

git status
Changes to be committed:
new file: docs/SWE573_Project_Development_Log_Iteration_1.pdf
```

This demonstrated the difference between an untracked file, a staged file, and a committed file.

```
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git log --oneline
9fdeb7d (HEAD -> main, origin/main, origin/HEAD) Add project goals to README
0587d5d Add initial project description to README
a32afbe Initial commit
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    docs/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> dir docs

Directory: C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment\docs

Mode                LastWriteTime         Length Name
----                -
-a---              9/29/2026   3:50 PM           580806 SWE573_Project_Development_Log_Iteration_1.pdf

PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git add docs/SWE573_Project_Development_Log_Iteration_1.pdf
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   docs/SWE573_Project_Development_Log_Iteration_1.pdf
```

Figure 9. The PDF moving from untracked content to a staged new file.

9. Amending a commit and observing the hash change

The documentation was first committed without an issue reference. Because the commit had not yet been pushed, git commit --amend was used to update the latest commit message to include (#2).

```
git commit --amend -m "Add project development log documentation (#2)"
```

The original short commit hash was ddb4fd4. After amending the commit message, the new hash became ef1b693. The file content did not need to change for the commit identity to change. This provided a concrete example that amending creates a new commit object rather than editing the existing commit in place.

```
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git commit -m "Add project development log documentation (#2)"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git log --oneline
ddb4fd4 (HEAD -> main) Add project development log documentation
9fdeb7d (origin/main, origin/HEAD) Add project goals to README
0587d5d Add initial project description to README
a32afbe Initial commit
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git commit --amend -m "Add project development log documentation (#2)"
[main ef1b693] Add project development log documentation (#2)
Date: Tue Sep 29 16:01:26 2026 +0300
1 file changed, 668 insertions(+)
 create mode 100644 docs/SWE573_Project_Development_Log_Iteration_1.pdf
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment> git log --oneline
ef1b693 (HEAD -> main) Add project development log documentation (#2)
9fdeb7d (origin/main, origin/HEAD) Add project goals to README
0587d5d Add initial project description to README
a32afbe Initial commit
PS C:\Users\Anil\Desktop\boun odevler\SWE573-SoftwareDevelopmentAssignment-gitfolder\SWE573-SoftwareDevelopmentAssignment>
```

Figure 10. git commit --amend changes the commit message and produces a new commit hash.

10. Connecting a Git commit to a GitHub Issue

The amended commit message included (#2). After pushing, GitHub recognized the issue reference and added the commit to Issue #2's activity timeline. This demonstrated how local Git history and

GitHub project-management records can be connected.

```
ef1b693 Add project development log documentation (#2)
```

A plain #2 reference links the commit to the issue without closing it. Closing keywords such as Closes #2 or Fixes #2 have different semantics and should only be used when the commit is intended to resolve the issue.

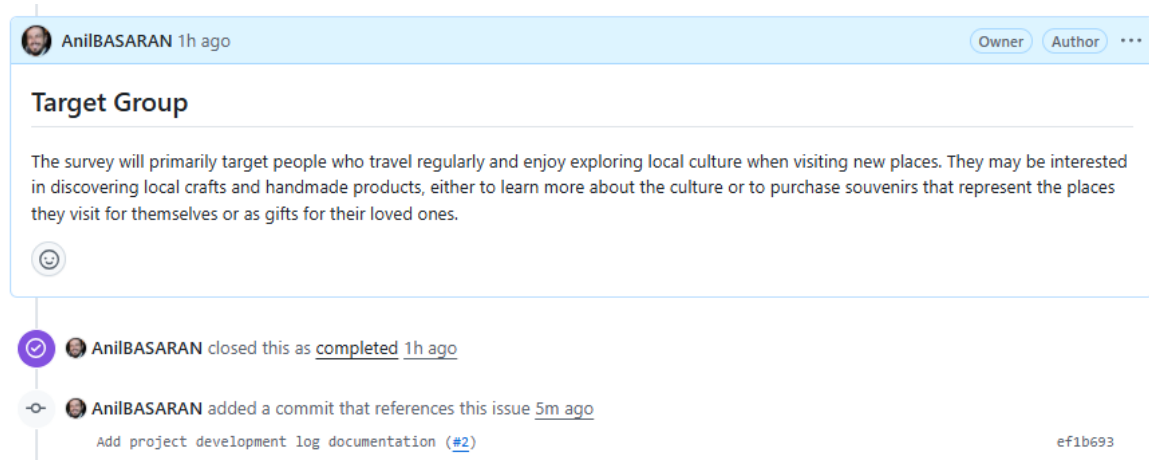


Figure 11. GitHub Issue #2 showing the pushed commit ef1b693 as a referenced commit in the issue timeline.

11. End-of-day project state

By the end of this iteration, the repository had a documented application concept, an expanded README, customized labels, a Project Planning milestone, two structured issues, a draft requirements-elicitation survey, and a version-controlled project development log.

```
#1 Define application concept -> completed
#2 Prepare potential user survey -> completed
Next: Conduct survey -> Analyze results -> Define initial requirements -> MVP
```

Key Git concepts practiced during the session included cloning, repository working directories, git status, git diff, staging, commits, short commit hashes, HEAD, main, origin/main, pushing, untracked files, git commit --amend, and GitHub issue references.

Repository recommendation

Store this document under docs/SWE573_Project_Development_Log_Iteration_1.pdf (or replace the previous Iteration 1 PDF with this final version). The Wiki should remain the living place for Git command notes and examples, while this PDF provides a dated snapshot of the work completed during the iteration.